# AI600 Deep Learning
# Programming Assignment 3
# Report

**Name:** Muhammad Danial Maqbool

**Student ID:** 25280029

**GitHub Repository:** https://github.com/danial-maqbool/25280029_AI-600_PA-3

## Analytical Questions

M. Danial Maqbool
2528 0029.

Deep Learning - PA3.

(1)

(1) No 1

$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}$ ; $W = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$ ; $b = 0$ ; $V = \begin{bmatrix} 1 & -1 & 2 & 0.5 \end{bmatrix}$ ; $c = 1$ ; $T = 3$.

(1a) $A = ReLU(X.W+b) = ReLU(Z)$

~~A = 3×3×1~~ and $X = 3 \times 3 \times 1$ ; Filter $W = 2 \times 2 \times 1, k = 1; P = 0; S = 1$

So output $Z = \left\lfloor \frac{X - W + 2P}{S} \right\rfloor + 1$.

(1) $Z = 2 \times 2 \times 1$.

Now,

$Z_{11} = XW + b = [(1.1) + (2.0) + (0.-1) + (1.1)] + 0 = 2$

$Z_{12} = [(2.1) + (0.0) + (1.-1) + (1.1)] + 0 = 2.$

$Z_{21} = [(0.1) + (1.0) + (1.-1) + (0.1)] + 0 = -1.$

$Z_{22} = [(1.1) + (1.0) + (0.-1) + (2.1)] + 0 = 3$

So,

Pre-activation feature map $\boxed{Z = \begin{bmatrix} 2 & 2 \\ -1 & 3 \end{bmatrix}}$

Now,

(2) $A = \max\left(0, \begin{bmatrix} 2 & 2 \\ -1 & 3 \end{bmatrix}\right) \Rightarrow ReLU(Z)$

$\boxed{A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}}$

(3) Flatten layer : $A_{flat} = [2 \ 2 \ 0 \ 3]^T$

(4) FC Layer : $Y = V.A_{flat} + C$

So,

$[1 \ -1 \ 2 \ 0.5]^T . \begin{bmatrix} 2 \\ 2 \\ 0 \\ 3 \end{bmatrix} + 1 = (2 - 2 + 0 + 1.5) + 1$

$\boxed{Y = 2.5}$

(5) Loss = $L = \frac{1}{2}(Y - T)^2 \Rightarrow L = \frac{1}{2}(2.5 - 3)^2 = \boxed{0.125 = L}$

(1b) Backward Pass:-

For V: $\frac{\partial L}{\partial V} = \frac{\partial L}{\partial Y} \cdot \frac{\partial Y}{\partial V}$ ; For W: $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Y} \cdot \frac{\partial Y}{\partial A_{flat}} \cdot \frac{\partial A_{flat}}{\partial A} \cdot \frac{\partial A}{\partial Z} \cdot \frac{\partial Z}{\partial W}$.

Now

$$\Rightarrow \frac{\partial L}{\partial Y} = \frac{\partial}{\partial Y}\left(\frac{1}{2}(Y-T)^2\right) = Y-T = \boxed{-0.5 = \frac{\partial L}{\partial Y}}$$

$$\Rightarrow \frac{\partial Y}{\partial V} = \frac{\partial}{\partial V}(V \cdot A_{flat} + C) = A_{flat} = \boxed{[2\ 2\ 0\ 3]^T = \frac{\partial L}{\partial V}}$$

$$\Rightarrow \frac{\partial L}{\partial V} = (-0.5)\ [2\ 2\ 0\ 3]^T = \boxed{[-1\ \ -1\ \ 0\ \ -1.5]^T = \frac{\partial L}{\partial V}}$$

$$\Rightarrow \boxed{\frac{\partial Y}{\partial A_{flat}} = V = [1\ \ -1\ \ 2\ \ 0.5]}$$

$\Rightarrow \frac{\partial A_{flat}}{\partial A}$; here we just reshape the (4x1) vector back to (2x2) matrix.

$$\Rightarrow \frac{\partial A}{\partial Z} = \begin{cases}(Z>0) = 1 \\ (Z \le 0) = 0\end{cases} \Rightarrow \boxed{\frac{\partial A}{\partial Z} = \begin{bmatrix}1 & 1 \\ 0 & 1\end{bmatrix}}$$

$$\Rightarrow \frac{\partial Z}{\partial W} = \frac{\partial}{\partial W}(X \cdot W + b) = X = \frac{\partial Z}{\partial W} = \begin{bmatrix}1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2\end{bmatrix}$$

So now,

$$\Rightarrow \cancel{\frac{\partial L}{\partial A_{flat}}(-0.5)} \cdot \frac{\partial L}{\partial A_{flat}} = \cancel{(-0.5)[-0.5\ \ 0.5\ \ -1\ \ 0}$$

$$= (-0.5)[1\ \ -1\ \ 2\ \ 0.5]$$

$$\frac{\partial L}{\partial A_{flat}} = [-0.5\ \ 0.5\ \ -1\ \ -0.25]$$

$$\Rightarrow \frac{\partial L}{\partial A} = \text{reshape}\left(\frac{\partial L}{\partial A_{flat}}, (2,2)\right) = \boxed{\begin{bmatrix}-0.5 & 0.5 \\ -1 & -0.25\end{bmatrix} = \frac{\partial L}{\partial A}}$$

$$\Rightarrow \frac{\partial L}{\partial Z} = \frac{\partial}{\partial Z}(\text{Relu}(Z)) \odot \frac{\partial L}{\partial A} = \begin{bmatrix}1 & 1 \\ 0 & 1\end{bmatrix}\begin{bmatrix}-0.5 & 0.5 \\ -1 & -0.25\end{bmatrix}$$

$$\boxed{\frac{\partial L}{\partial Z} = \begin{bmatrix}-0.5 & 0.5 \\ 0 & -0.25\end{bmatrix}}$$

$$\Rightarrow \frac{\partial L}{\partial W} = X \cdot \frac{\partial L}{\partial Z} = \boxed{\begin{bmatrix}0.25 & -1.25 \\ 0.5 & -0.5\end{bmatrix} = \frac{\partial L}{\partial W}}$$

So finally:-

$$\frac{\partial L}{\partial V} = \begin{bmatrix} -1 & -1 & 0 & -1.5 \end{bmatrix}^T$$

and

$$\frac{\partial L}{\partial W} = \begin{bmatrix} 0.25 & -1.25 \\ 0.5 & -0.5 \end{bmatrix}$$

(1c) a) What weights changes the most and in what direction?

→ From the gradient $\frac{\partial L}{\partial W}$, we can see that the weight $w_{12}$ has the largest gradient so we know that it will change the most.

→ The weight will change in positive direction i.e its value will be increased because we know that:-

$$w_{new} = w_{old} - \alpha . \frac{\partial L}{\partial w}$$

and as $\frac{\partial L}{\partial w_{12}}$ is negative $(-1.25)$, so we will be increasing the value of $w_{old}$ by adding the factor $(+1.25\alpha)$.

b) Structural Explanation:-

Decision for the weight that changes the most is dependent on the highest gradient and highest gradient depends on two factors, highest input value and most error given. So we can see that $A_{12} = 2$ and $A_{22} = 3$ are highest inputs but $\frac{\partial L}{\partial A_{12}} = 0.5$ and $\frac{\partial L}{\partial A_{22}} = -0.25$.

So overall we can see that $w_{12}$ has the most effect and changes the most.

Q NO 3

(3a) After replacing Max Pooling with Flatten and FC layer the architecture becomes:-

$$\text{input } X \ (3\times3) \longrightarrow \boxed{\text{Conv } 2\times2 + \text{ReLU}} \xrightarrow{\text{Feature Map}} A \ (2\times2) \longrightarrow \boxed{\text{Max Pooling}} \longrightarrow \text{Prediction } Y.$$

Now,

$A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$ and $\boxed{Y = 3.}$ (Forward Pass)

$$L = \frac{1}{2}(Y-T)^2 = \frac{1}{2}(3-3)^2 = 0$$

Back ward Pass:-

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial Y} \cdot \frac{\partial Y}{\partial A} \cdot \frac{\partial A}{\partial Z} \cdot \frac{\partial Z}{\partial W}$$

Now,

$\frac{\partial L}{\partial Y} = Y - T = 0$ ; $\frac{\partial Y}{\partial A} = \begin{bmatrix} 0 & 0 \\ 0 & 3 \end{bmatrix}$ ∴ all inputs except the max one becomes 0. considering $\epsilon = 0$

$\frac{\partial A}{\partial Z} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ ; $\frac{\partial Z}{\partial w} = X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}$

Since loss $= 0$ ; $\frac{\partial L}{\partial w} = 0$

if the loss was not zero we can consider that in max pooling the gradient sends back the value of only the "winning" pixel. that had max value. For all other position, the gradient gives a 0. So for our case, the max value was 3 ($A_{22}$), hence only four pixels ($X_{22}, X_{23}, X_{32}, X_{33}$) recieve any gradient. so all other pixels will have gradient of 0. i.e $\begin{bmatrix} 0 & 0 & 0 \\ 0 & X_{22} & X_{23} \\ 0 & X_{32} & X_{33} \end{bmatrix}$

## 3b) Normalization:-

The new architecture would be:-

$$\text{Input } X \longrightarrow \boxed{\text{Conv } 2\times2 + \text{ReLU}} \xrightarrow{A} \boxed{\text{Normal-ization}} \xrightarrow{A_{norm}} \boxed{\text{Flatten}} \xrightarrow{A_{flat}} \boxed{\text{FC Layer}} \longrightarrow \text{Prediction } Y.$$

Now using $A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$

$$\text{Mean} = \mu = \frac{2+2+0+3}{4} = 1.75$$

$$\text{Variance} = \sigma^2 = \frac{(2-1.75)^2 + (2-1.75)^2 + (0-1.75)^2 + (3-1.75)^2}{4}$$

$$\boxed{\sigma^2 = 1.1875}$$

$$\text{Standard Deviation} = \sigma = \sqrt{1.1875} \approx 1.09$$

Now,

$$\text{Normalized feature map} = A_{norm} = \frac{A - \mu}{\sigma}$$

$$= \begin{bmatrix} \frac{2-1.75}{1.09} & \frac{2-1.75}{1.09} \\ \frac{0-1.75}{1.09} & \frac{3-1.75}{1.09} \end{bmatrix} = \boxed{\begin{bmatrix} 0.23 & 0.23 \\ -1.61 & 1.15 \end{bmatrix} = A_{norm}}$$

## 3c) Mathematical explanation of translation invariance:

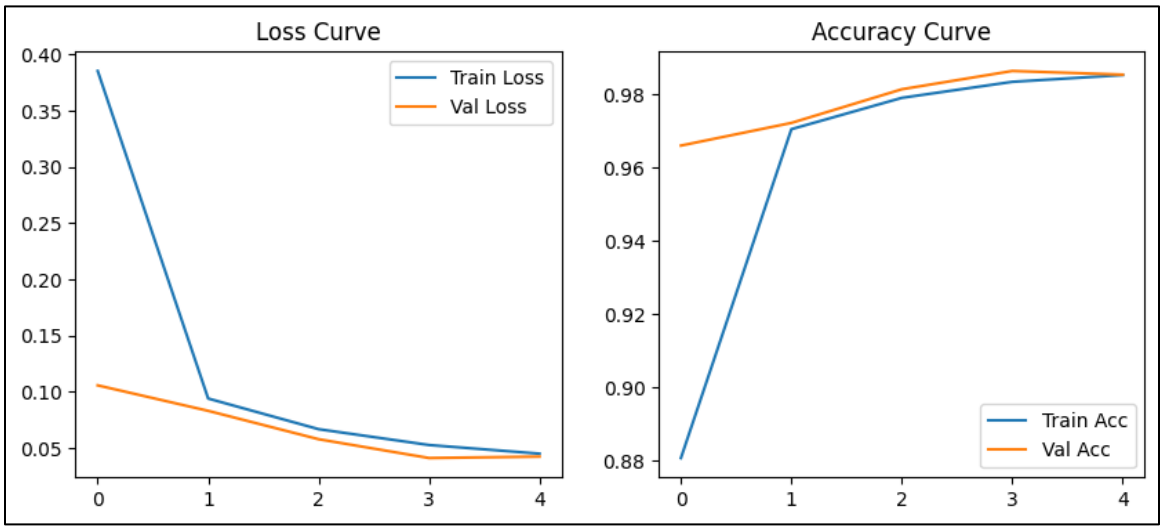
if input image shifts slightly to left or right, the specific 2x2 patch of pixels that generated the max activation will just be processed at a different step of convolution.

This means that value 3 will simply move to a different spatial location inside the matrix (i.e from $A_{22}$ to $A_{21}$) because global max pooling simply look for single max value from the whole matrix, regardless of position so the Y would still have been 3, keeping final prediction and loss always the same.

# Programming Questions

## Task 1: Custom CNNs and Shortcut Learning

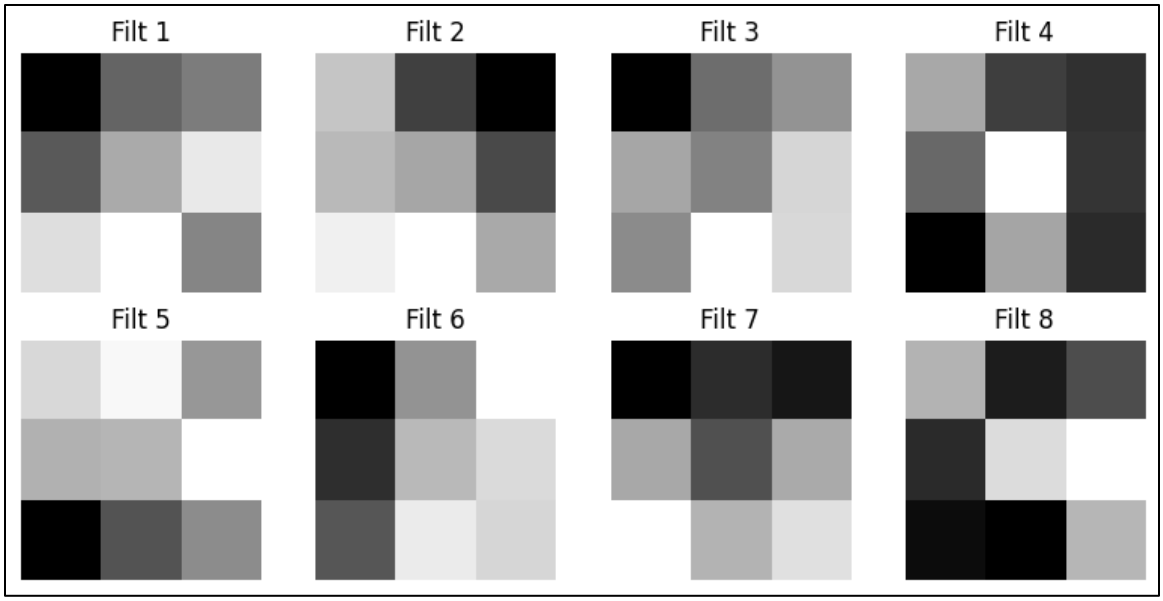
### Part A: Standard MNIST




*Analytical Question 1.1:*

**Examine your loss curves. Did your model overfit, underfit, or generalize well?**

I believe the model generalized well, this is because we can see that the training loss and validation loss curves are decreasing more or less together and there is no big gap between the loss and accuracy curves of the training and validation values. If the model had overfit there should've been a huge gap between the training and validation values. If the model had underfitted then the accuracy would not have been so high. So, I believe that the model has generalized well.




*Analytical Question 1.2:*

**Can you visually identify any rudimentary edge or texture detection occurring?**

Yes, we can see some edge detection through the filters. The contrasting colours of pixel from bright to dark show some edge detection. For example, filter 1 and 2 shows somewhat diagonal orientation while filter 6 shows horizontal edge detection. These early layers are capturing basic structural changes.

## Part B: Colored-MNIST (C-MNIST)

*Analytical Question 1.3:*

**Report the accuracy of both test sets. You should observe a significant drop in performance on the unbiased test set. Explain this phenomenon mathematically and conceptually.**

The model showed high accuracy on the biased test set (99%) and much lower accuracy on the unbiased test set (87.5%). I think this happened because in biased set, the model learned and started to classify the numbers based on the background colours rather than the actual shape. The observed colours were not found in un-biased set and hence this 'trick' had failed and caused the accuracy to be lower.

**Why did the optimizer prefer the color feature over the shape feature during training?**

The main aim of neural networks is to minimize the loss function which they do by calculating gradients to find the fastest path of minimization. To map a flat, uniform RGB value directly to a class label, we require very little mathematical adjustments in the early

weights as compared to learning complex spatial arrangement of edges, thus the optimizer selects the route of color features, which is mathematically easier.

### *Analytical Question 1.4:*

**Propose training strategies that could force the network to ignore the color and focus on the shape of the digits**

We can convert 3 layered RGB inputs to 1 layered greyscale image to remove the color features before the network sees it. We can also use data augmentation by changing the colours of everything in the images randomly during training phase. This stops the model from relying on mathematical correlations between the colours and labels and forces it to learn the recognition of digits.

## Task 2: Transfer Learning and Interpretability

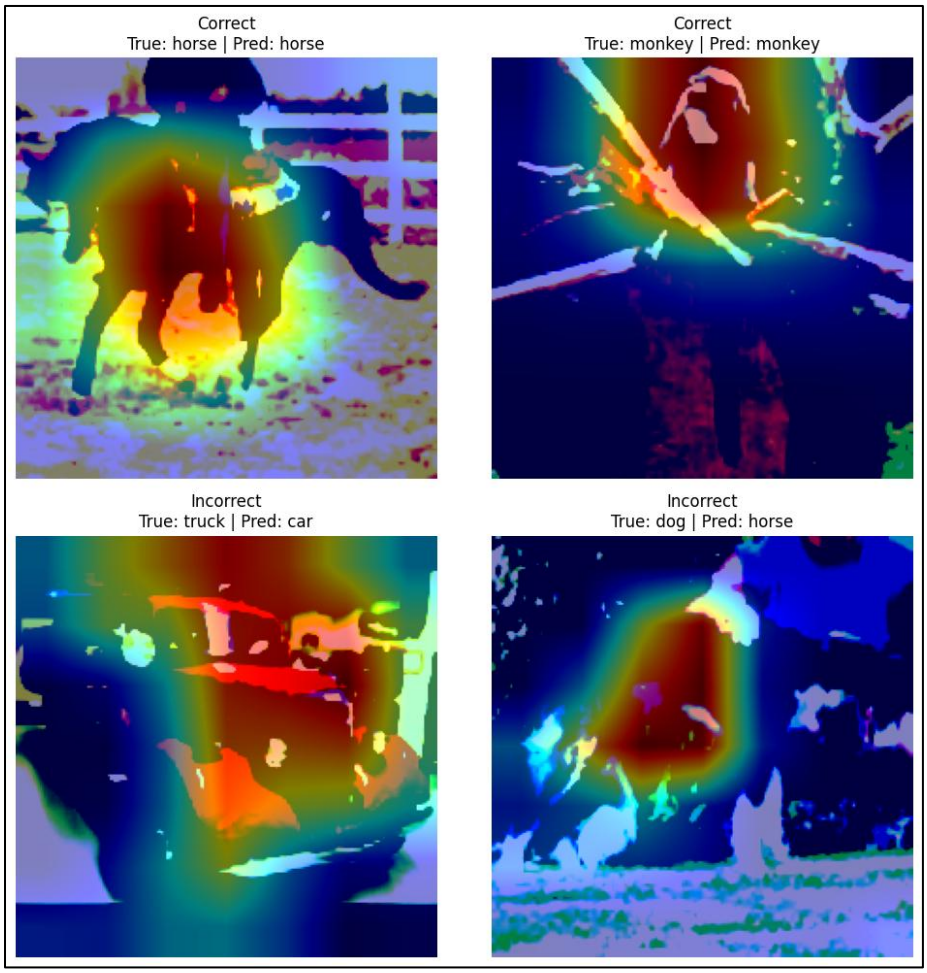
### Part A: Fine-tuning ResNet-18

***Analytical Question 2.1:***

**Why freeze early layers? What features are stored in early layers?**

Computationally: Freezing the early layers means that the network does not calculate gradients for millions of parameters. This reduces VRAM usage and speeds up training time. Also, the STL-10 dataset is relatively smaller in size and if we don't freeze the early layers and allow all the weights to be updated, this can make the model start memorizing the data and cause overfitting. Also, if we see this in terms of learning, then early layers store low-level filters like edges, textures, etc. so we keep these things as it is and use them to learn more complex, class-specific concepts for our context.

### Part B: Visualizing Decisions with GradCAM



### *Analytical Question 2.2:*

**Correct Predictions**

The GradCAM heatmaps for the images that were correctly classified show that the model is mostly looking at the main object and not the background. The bright red and yellow areas of the heatmap are right over the object's defining features, while the cooler blue areas show the background. This shows that they didn't have much of an effect on the decision.

### *Analytical Question 2.3 (Incorrect Predictions):*

From the heatmap for incorrectly classified images, we can see why the model was confused and made mistakes. The highlighted areas show that the model either used a false correlation (by focusing too much on a background element that is closely linked to another class) or it mixed up a localized edge feature on the object with the defining feature of the wrong class.

## Generative AI Policy Compliance

I made use of AI (Gemini 3 Pro) to refresh some theory concepts and debug my code along the way. I used AI to discuss hyperparameter tuning strategies such as configuring the OneCycleLR scheduler and label smoothing, to maximize efficiency under the parameter limit, and to generate the boilerplate matplotlib code for plotting the loss curves and confusion matrix. However, all core architectural decisions, training runs and final evaluations/analysis were executed and verified on my local hardware.